PRÁCTICA GUIADA

PARTE 1

SHEILA CERQUEIRAS CASTRO

1°DAW

ÍNDICE

Actividad 1:.....3
Actividad 2:.....3
- ¿En qué lenguaje de programación está escrito?.....3
- ¿Es un lenguaje compilado o interpretado?.....3
- ¿Has podido ejecutarlo directamente nada más instalar el Visual Studio Code?.....3
- ¿Qué tienes que instalar para ejecutarlo?.....3
- ¿Has tenido que instalar algún plugin adicional?¿Cuál?.....4
Actividad 3:.....4
1.- Búsqueda de información.....4
2- Instalación plugin.....5
Actividad 4: Depuración de código (modo Debug) y puntos de ruptura (breakpoints).....7
4.1.- Pon un punto de ruptura en el script_1.py.....7
4.2.- Ahora ejecuta el script_2.py.....12
4.3.- Ejecuta el script_3.py en modo normal.....16
4.4.- Ejecuta el script_4.py.....18
¿Se produce algún error?¿Por qué? Explica que está ocurriendo.....18
4.5.- Ejecuta el script_5.py paso a paso.....19
4.6.-Ejecuta el script_6.py.....21
4.7.- Ejecuta el script_7.py en modo normal para entender qué hace el programa.....25
Prueba a poner puntos de ruptura y ejecutar el código línea a línea para dar con el error y solucionarlo.....27

Actividad 1:

Ya tenía el visual studio code instalado anteriormente, por lo que no he necesitado descargarlo.

Actividad 2:

En esta actividad los scripts que usamos son: “DepurarParte1.zip.”.

1. Descargue los archivos correspondientes, al abrir el script1 de la carpeta de “DepurarParte1”, me pidió que descargue la instalación de python ya que los script estaban escritos en ese lenguaje de programación.

<u>DepurarParte1.zip:</u>

- **¿En qué lenguaje de programación está escrito?**

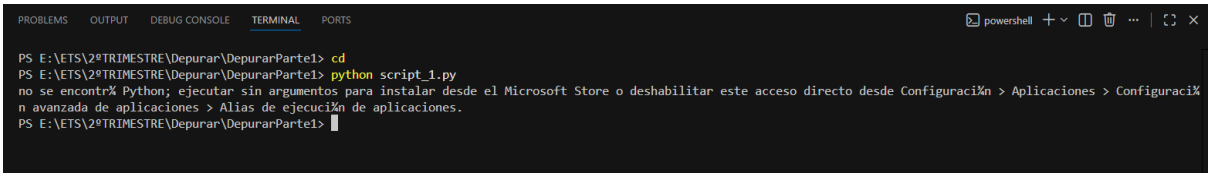
En python

- **¿Es un lenguaje compilado o interpretado?**

Es un lenguaje interpretado ya que estamos utilizando python, este es un lenguaje de programación compilado porque traduce y ejecuta el código línea por línea en tiempo de ejecución

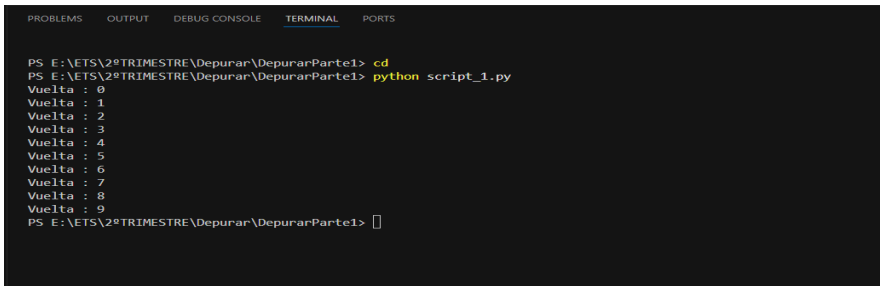
- **¿Has podido ejecutarlo directamente nada más instalar el Visual Studio Code?**

No, he intentado abrir el archivo desde una nueva terminal y no me lo permite, me pedia que instalara la aplicacion de python:



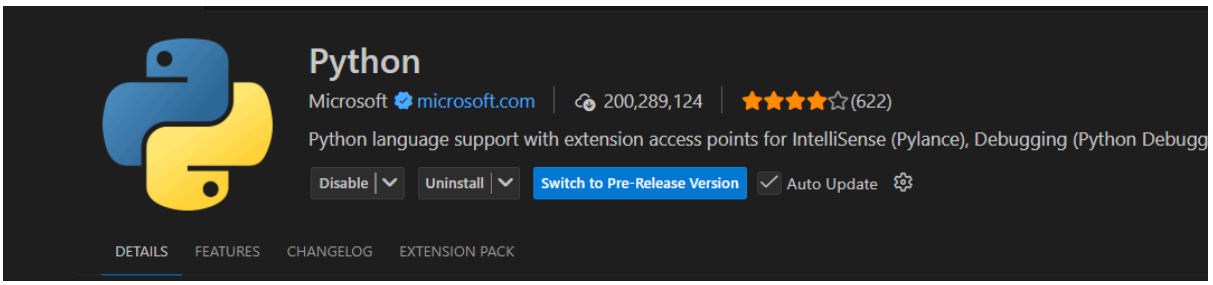
- **¿Qué tienes que instalar para ejecutarlo?**

He instalado el python y he vuelto a colocarlo en una nueva terminal, los mismos comandos pero esta vez, sí me ha funcionado

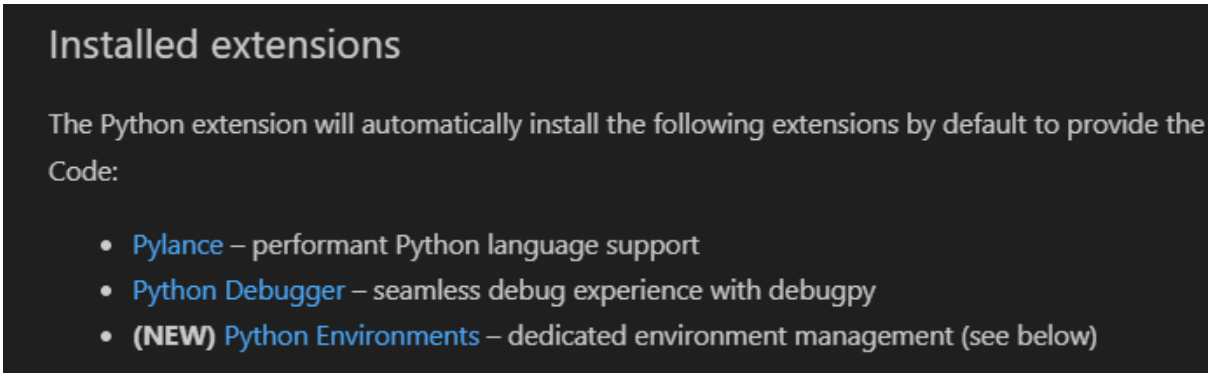


- **¿Has tenido que instalar algún plugin adicional?¿Cuál?**

Sí, me ha pedido instalar un plugin de python cuando recién abrí el archivo. Porque el visual code ha analizado que el archivo era de python y para darle support a este tipo de ficheros, me recomendó descargar este plugin que me instala a su vez, un paquete de extensiones para el lenguaje de python



Estas fueron las extensiones que me descargó automáticamente al instalar el plugin python:



Actividad 3:

1.- Búsqueda de información

- **Principales características de python**
  - Es un lenguaje interpretado, no compilado, usa tipado dinámico, fuertemente tipado.
  - Es multiplataforma, lo cual es ventajoso para hacer ejecutable su código fuente entre varios sistema operativos
  - Es un lenguaje de programación multiparadigma, el cual soporta varios paradigmas de programación como orientación a objetos, estructurada, programación imperativa y, en menor medida, programación funcional.
  - En Python, el formato del código es estructural.

- **Puntos fuerte de python**
  - Lenguaje sencillo y de gran potencial

Python no tiene tanta abstracción como otros lenguajes de programación, por lo que su aprendizaje es más intuitivo

  - Bibliotecas y Frameworks

Como Python es de código abierto, permite que más personas puedan contribuir en el desarrollo de código y frameworks que ayudan a otros programadores a crear proyectos.

  - Multiplataforma

Puede ser ejecutado en cualquier sistema operativo en el cual se opere

- **Puntos débiles de python**
    - Procesamiento lento

Procesar de forma lenta, ya que no tiene definido un uso específico y compilado

    - No es perfecto para todo

En el caso del desarrollo de aplicaciones móviles, no es la mejor opción para programar, ya que existen otros lenguajes más especializados y dan mejores resultados.

    - Consumo de memoria

La cantidad de datos que maneja el lenguaje es muy extensa, por lo que su consumo de memoria es excesivo, y a menos que tengas un equipo con gran capacidad, no es lo más indicado para usar en proyectos grandes.

- **Principales campos de aplicación a día de hoy en el mundo de las tecnologías de la información**

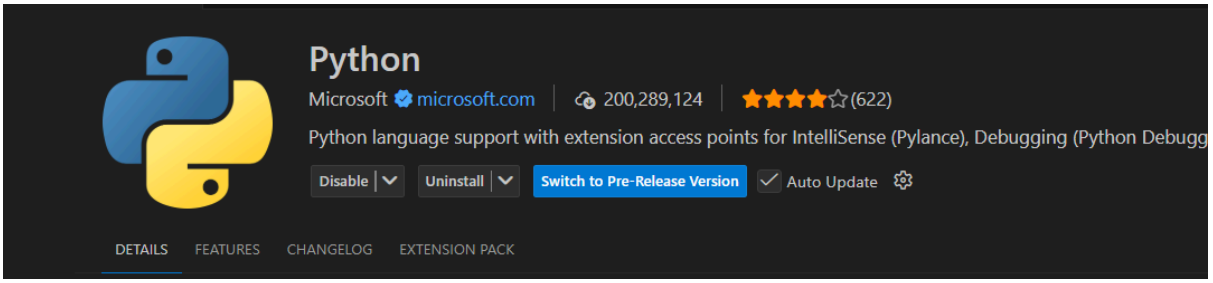
Los campos principales del uso de Python hoy en día son: Big data analytics, Data mining, Blockchain, Machine learning, Desarrollo web, Juegos y gráficos 3D

2- Instalación plugin

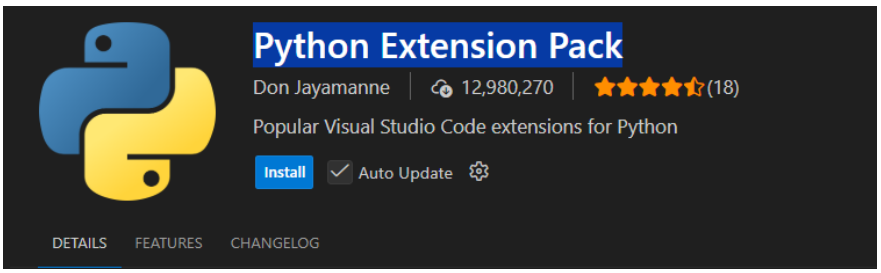
Busqué en google, cuales eran los mejores plugin de python para mejorar la eficiencia. La más recomendada era python, pero esa ya la había instalado previamente cuando abrí el archivo así que seguí buscando por más. Después coloqué python en las extensiones de vscode y me salió un pack de extensiones de python que me añade varios plugin más que sirven para la eficiencia de python.
Pero como uno me salía que estaba obsoleto, decidí desinstalarlo, se llamaba “Python Environment Manager (deprecated)”.

- **¿Cuál o cuáles has instalado?**

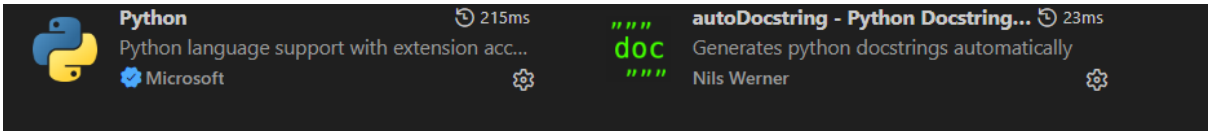
1. Python:



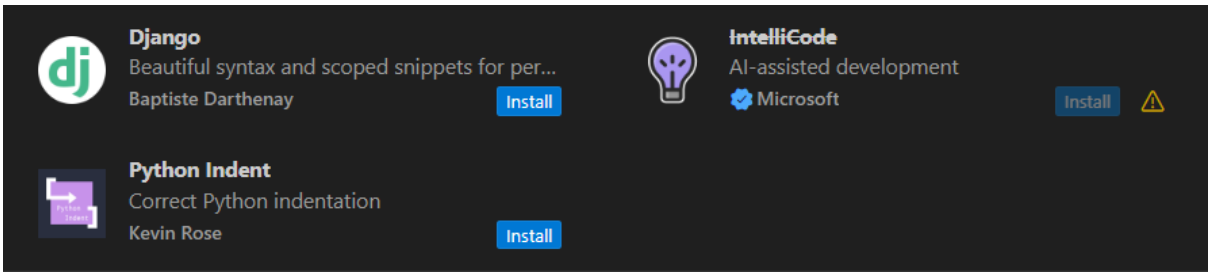
2. Python Extension Pack:



3. autoDocstrins



4. Django
5. IntelliCode
6. Python Indent



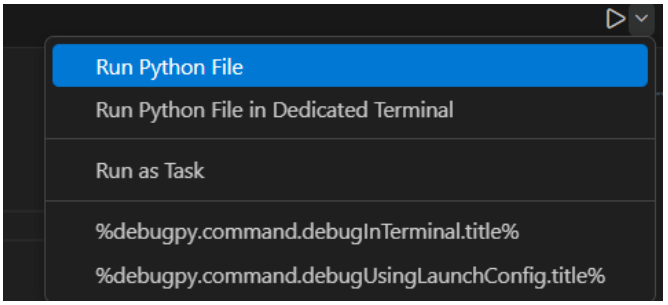
- **¿Para qué sirven?**
  - Python Extension Pack: plugin que me descarga un paquete de extensiones para mejorar el uso de python aparte de instalar el python en sí.
  - Python Indent: sirve para corregir la sangría de Python en Visual Studio Code.
  - Python Doctring Generator: sirve para generar rápidamente docstrings para funciones de python porque te permite genera rápidamente un fragmento de docstring, te deja elegir entre varios tipos de formatos de docstrings, interfiere los tipos de parámetros a través de pistas de tipo pep484, valores por defecto y nombres var, da soporte para args, kwargs, decoradores, errores y tipos de parámetros.
  - Django: es un framework web de Python de alto nivel que fomenta el desarrollo rápido y un diseño limpio y pragmático. Se encarga de gran parte de las complicaciones del desarrollo web, para que puedas concentrarte en escribir tu aplicación sin tener que reinventar la rueda. Además, es gratuito y de código abierto.
  - IntelliCode: está ya obsoleto porque el github copilot en vscode ya hace su función pero consiste en implementar la IA en tu código como un asistente a la hora de programar.

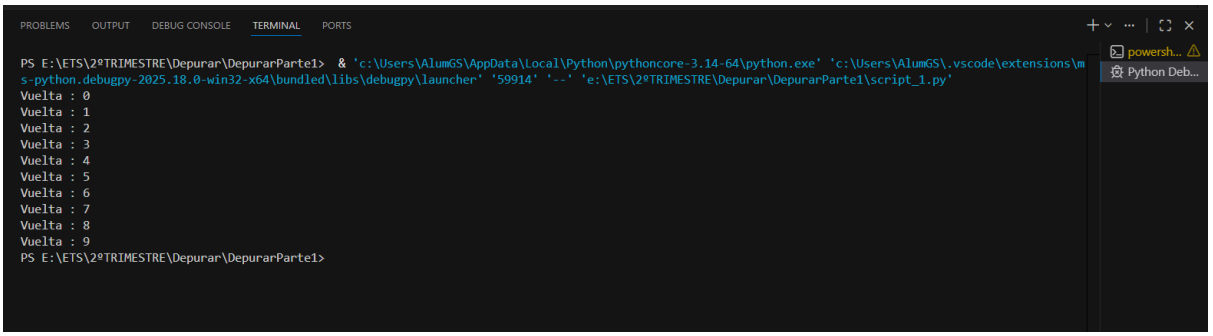
Actividad 4: Depuración de código (modo Debug) y puntos de ruptura (breakpoints)

4.1.- Pon un punto de ruptura en el script_1.py.

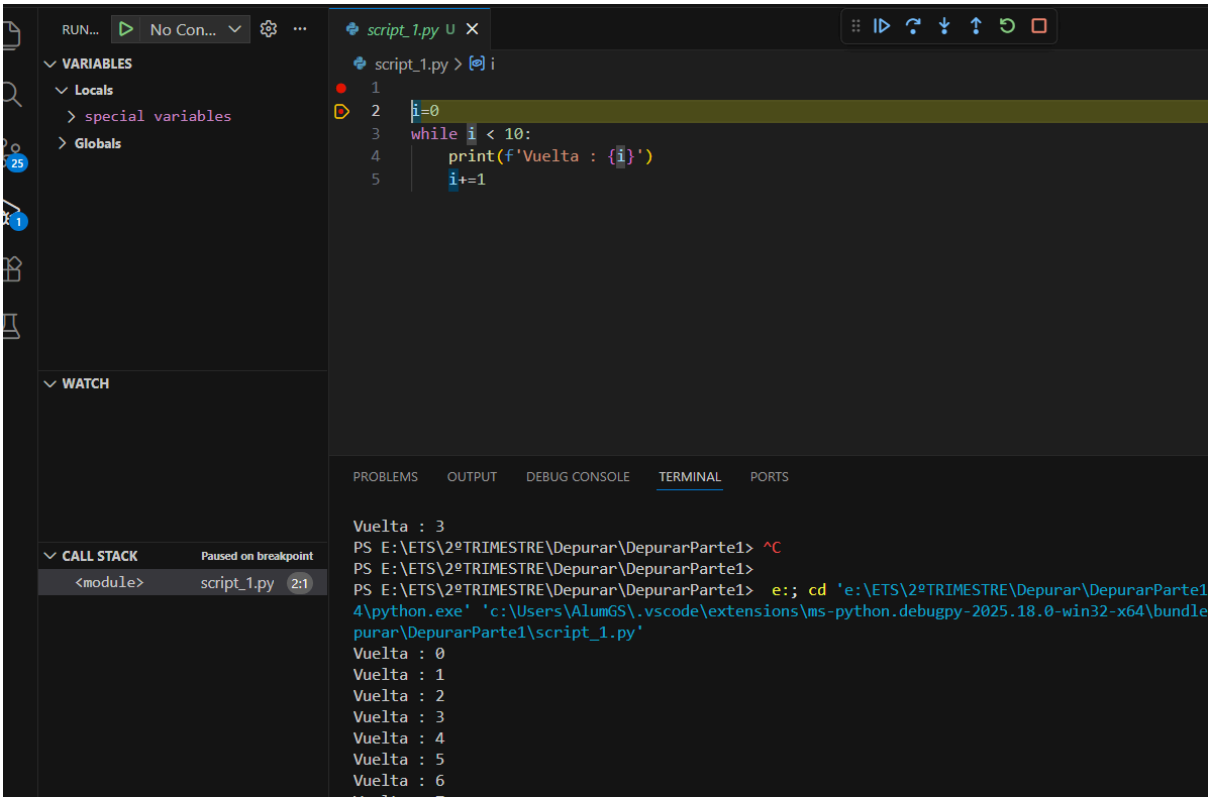
Donde creas conveniente y ejecuta línea a línea viendo como se va ejecutando el código y los valores que van imprimiéndose por pantalla. Prueba con las 4 opciones de ejecución (continue, step over, step into, step out). ¿Qué diferencias hay entre los 4 modos de ejecución para este caso?

1. Me ubico en el fichero de script1. Ejecuto el programa en modo debug, elijo la primera opción de debug en terminal, y ya me aparece la terminal en modo debug de python:

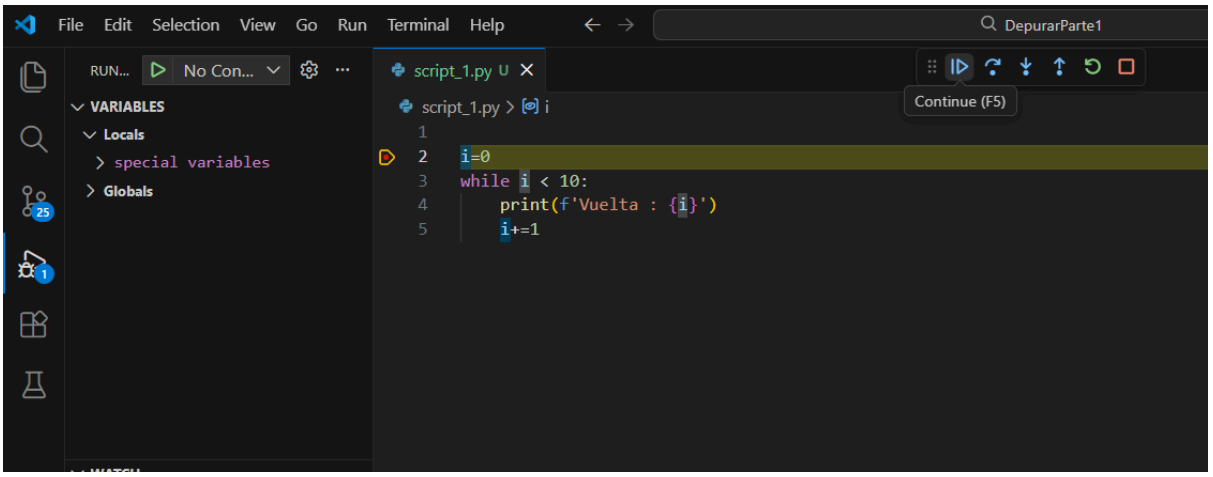




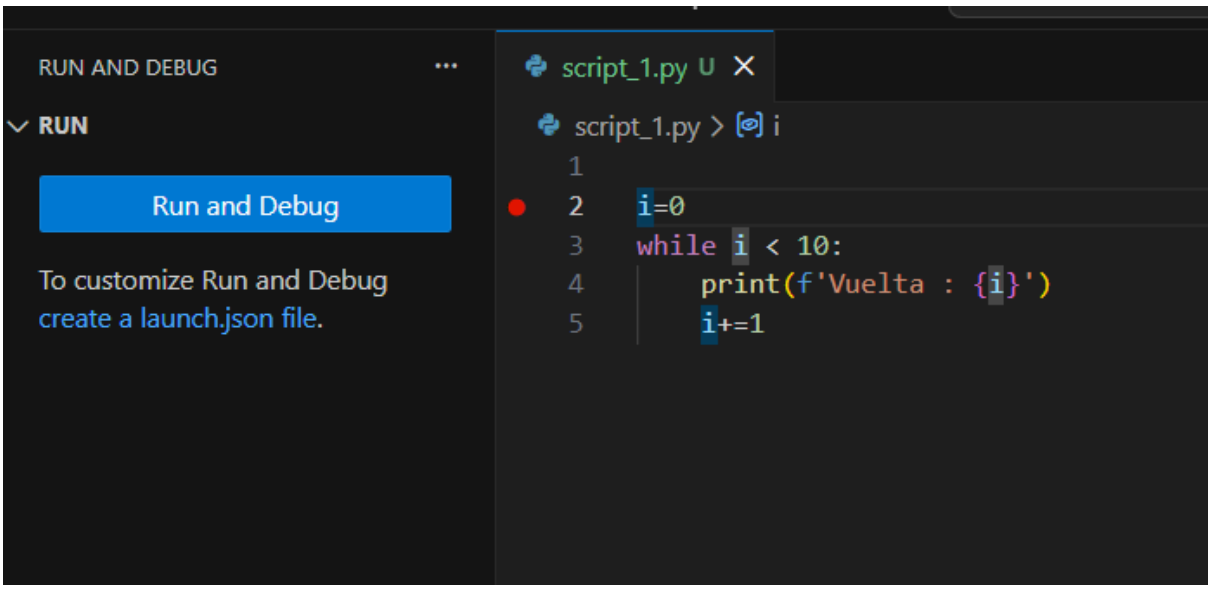
2. Al tocar una de las líneas, colocando un punto rojo en dicha línea de código hacemos lo que llamamos breakpoint, le damos a debug, me muestra los valores de las variables que hay dentro del breakpoint:

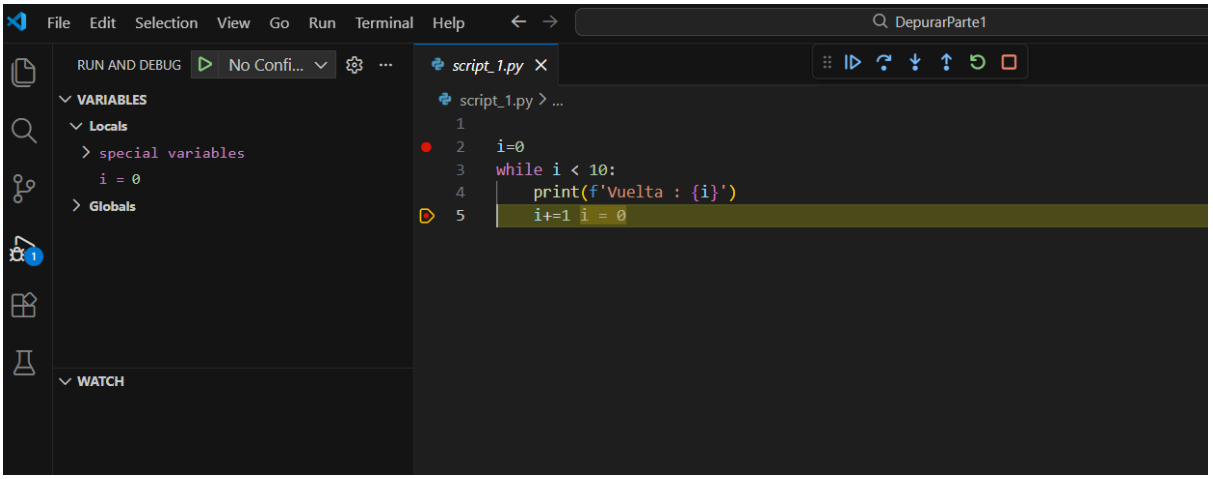


3. Ahora pruebo con las 4 opciones de ejecución que pide el ejercicio:

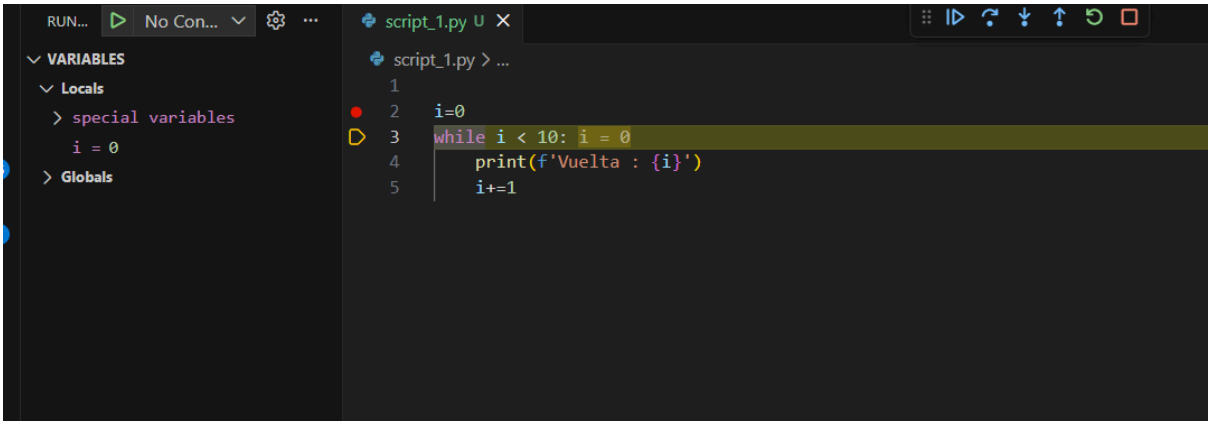


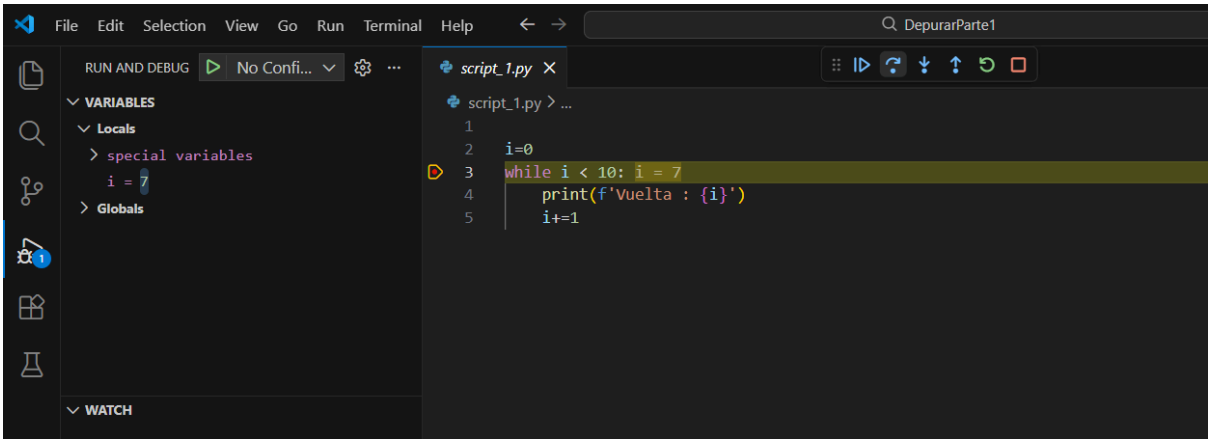
3.1. <u>Continue</u>: Me para el debug que estoy haciendo y me saca de él pero no me quita el breakpoint que he colocado en la línea 2. Esto es porque no coloqué otro breakpoint. Si coloco otro breakpoint, en la línea 5, al usar continue, salta de un breakpoint al otro, es decir, de la línea 2 a la 5.

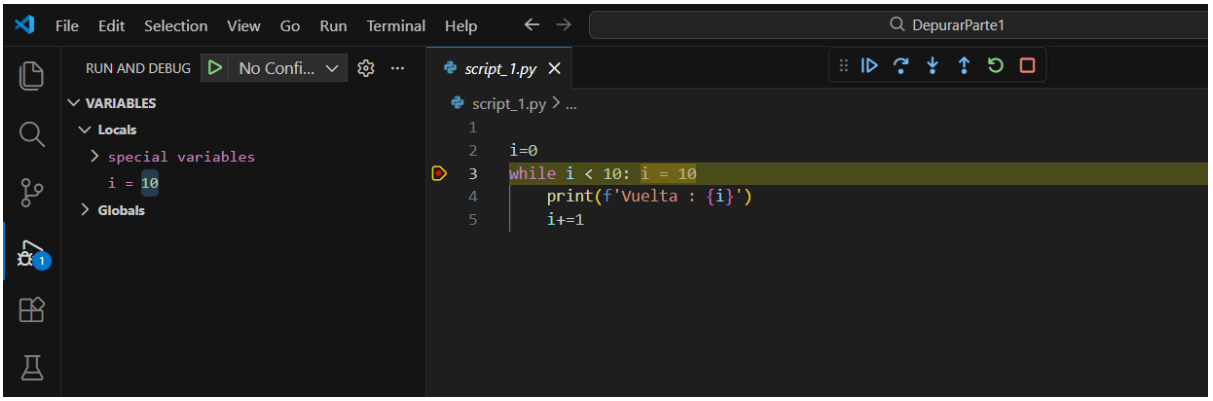


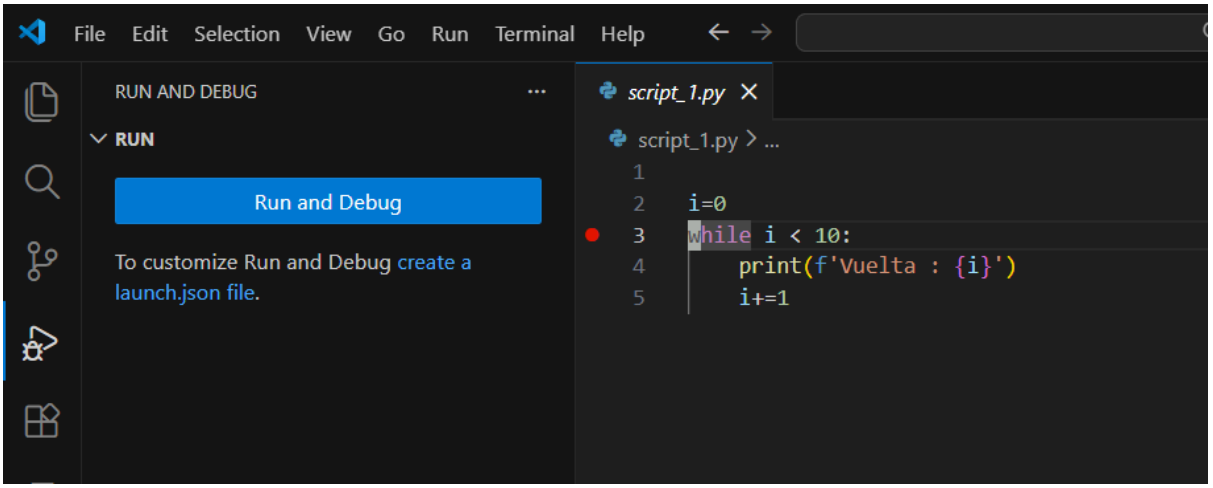


3.2. <u>Step over:</u> Pasa el breakpoint a la siguiente línea y analiza las variables de la siguiente línea que ahora se ve marcada (1º captura). Si colocamos el breakpoint en una función como la de la línea 3, al usar step over, se ejecutará la función cada vez que complete una vuelta del while, y podemos ver como incrementa la "i" por cada ciclo completo de la función (como vemos en la 2º captura). Y cuando llega la función while a "i=10" (3º captura), se para de ejecutar el debug y te saca de él, porque da por terminada la función ya que así está determinado en el while (4º captura).

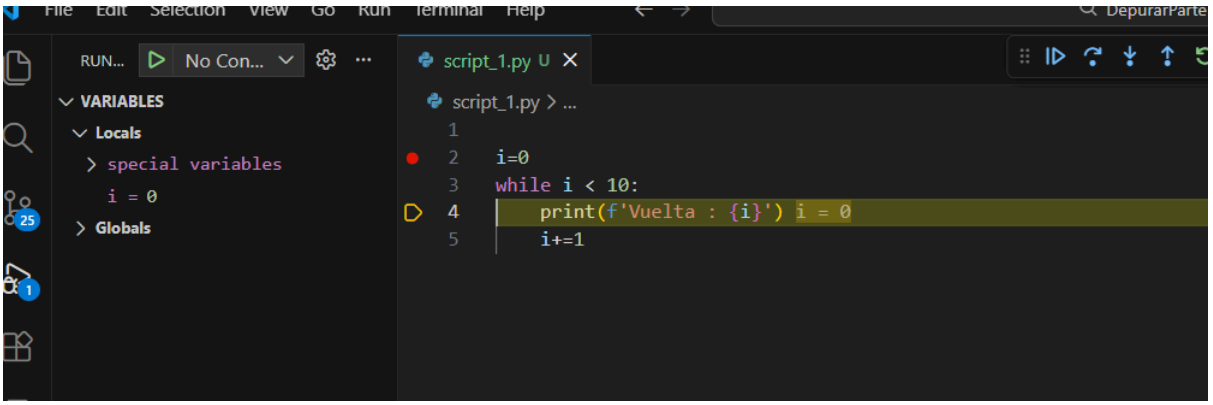




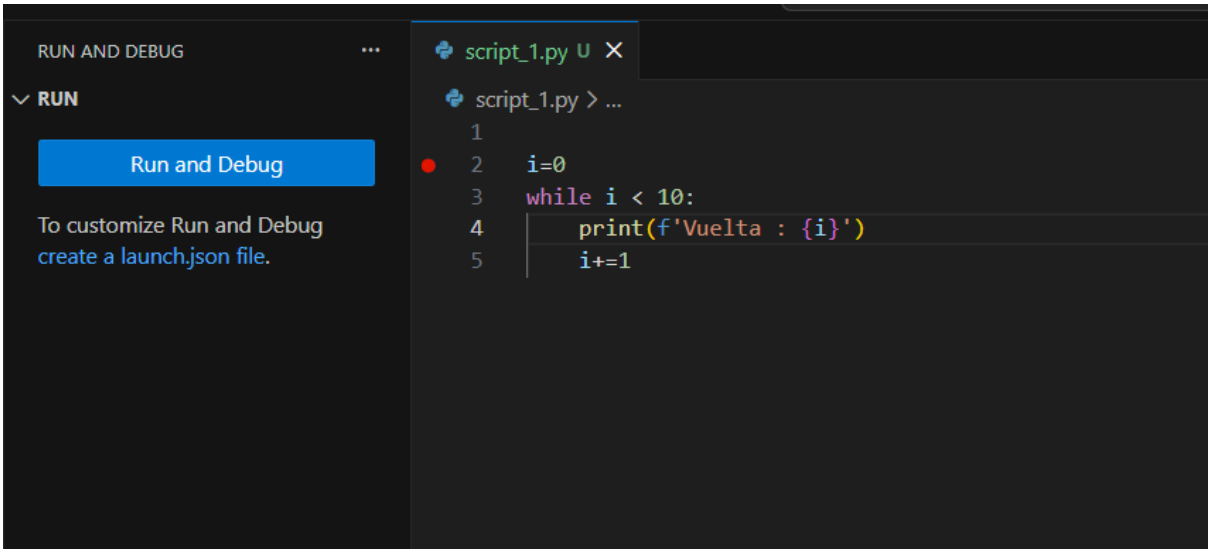


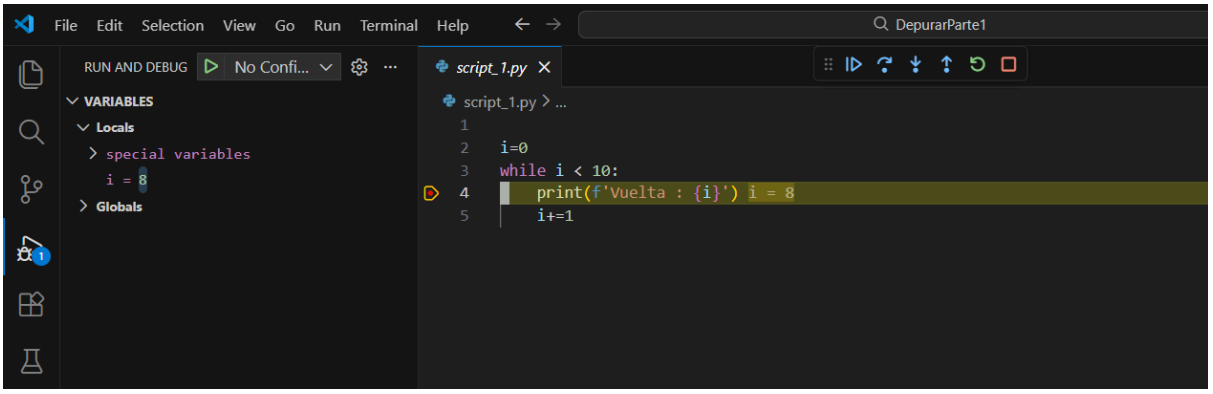


3.3. Step into: va ejecutando cada línea cada vez que la usas pero cuando ejecuta una función como while, se mete dentro de la función while y analiza esas variables y no sale de la función hasta que termine la función en la que está. Cuando ha terminado el while, da por finalizado el debug.



3.4. <u>Step out:</u> si usas el step out cuando no estás dentro de una función como en la línea 2, entiende que quieres terminar el debug, y te saca directamente. Si colocas el breakpoint dentro de una función como la del while (captura 2), cada vez que usas el step out, incrementa el valor de “i” sin tener que dar una vuelta al while, es decir, no sale de la línea del breakpoint (línea 4) hasta que llega a 9, y da por terminada la función y el debug.





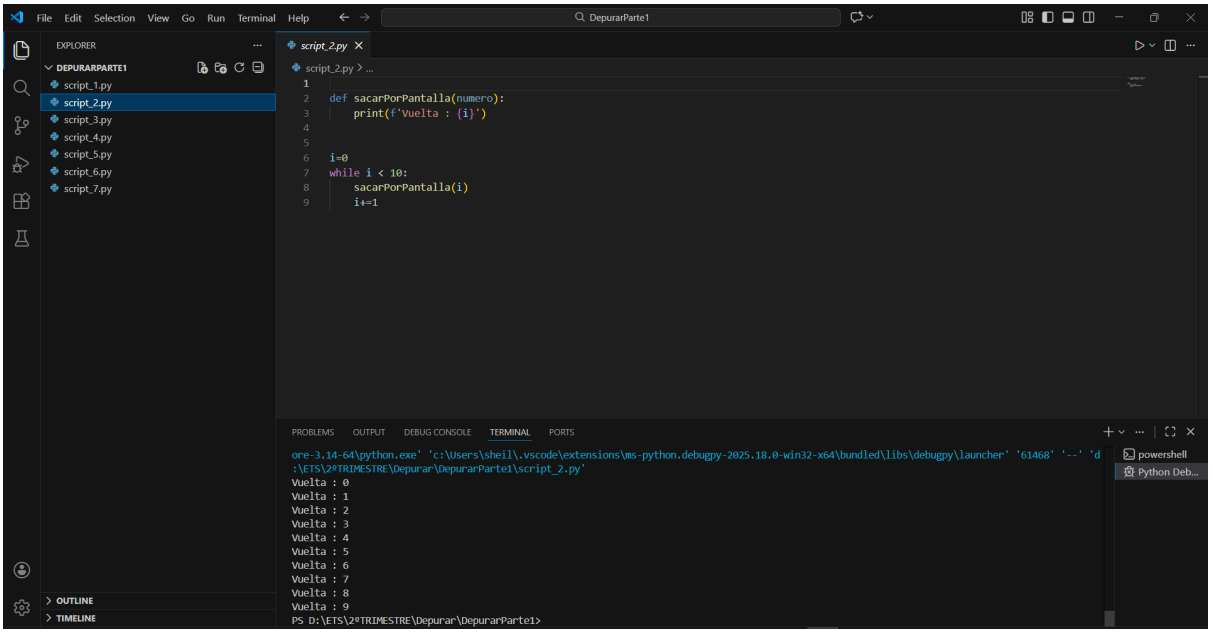
Las diferencias que podemos ver en este caso para las 4 opciones de ejecución en el debug, es que continue salta de un breakpoint a otro directamente, step out es la única que sí está el breakpoint dentro de una línea de la función, no deja esa línea hasta que acabe la función sin la necesidad de ejecutar todo la función para incrementar la i como, en cambio,

tienen que hacer el step into y el step over. Y la última diferencia es que step over no entra dentro de la función sino que va ejecutando cada línea, mientras que step into, sí entra dentro de la función.

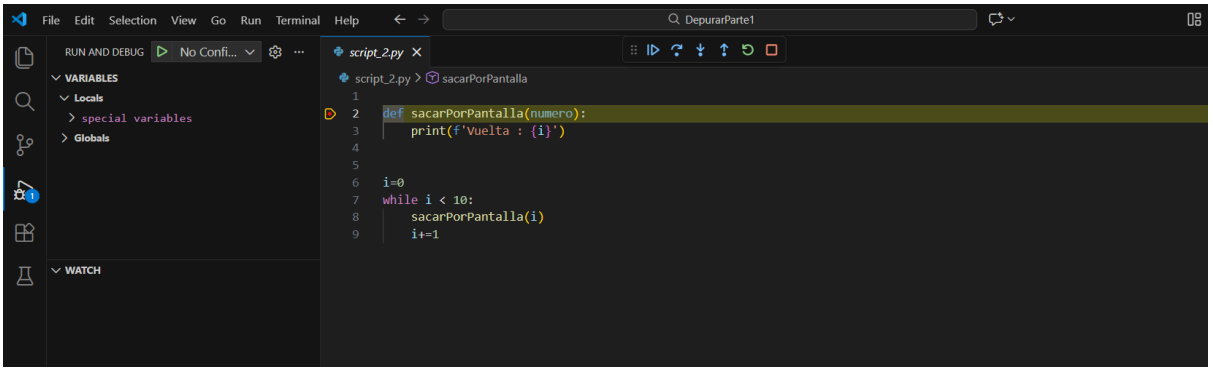
4.2.- Ahora ejecuta el script_2.py.

Coloca nuevamente al menos un punto de ruptura. Prueba con las 4 opciones de ejecución (continue, step over, step into, step out). ¿Que diferencias hay entre los 4 modos de ejecución? (Saca capturas de pantalla dónde se vean las diferencias si es que las hay)

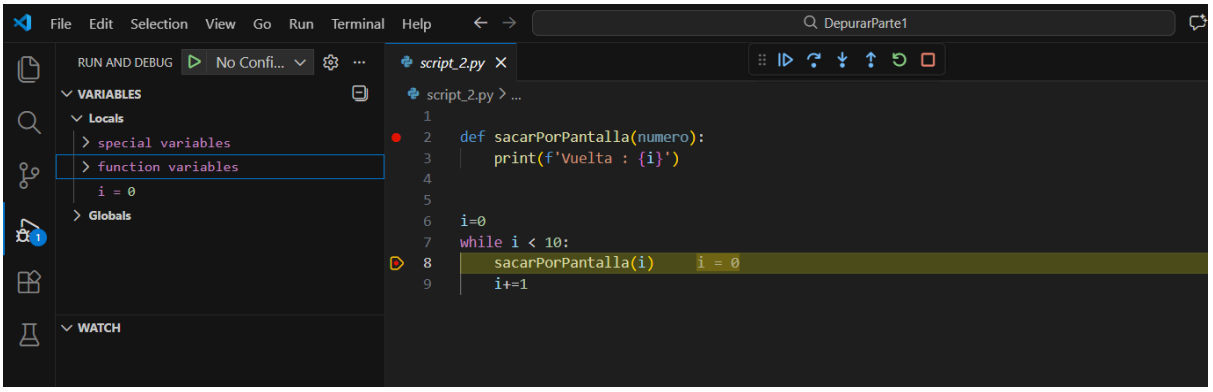
1. Primero ejecuto el script2:

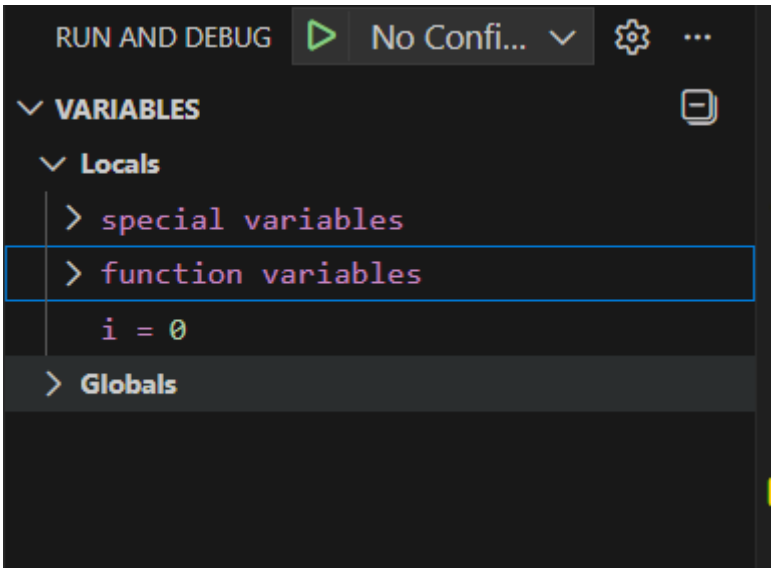


2. Coloco un breakpoint en la línea 2 y comienzo a probar las 4 opciones de ejecución:

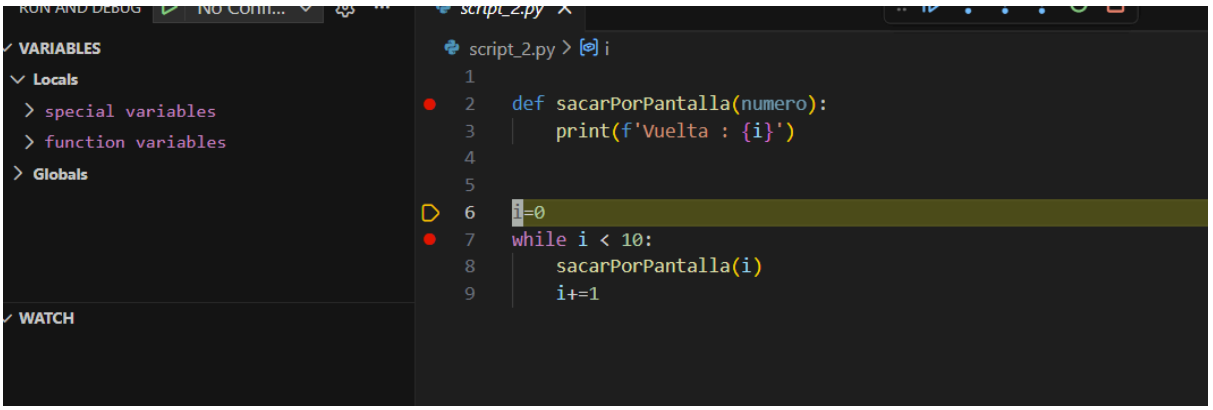


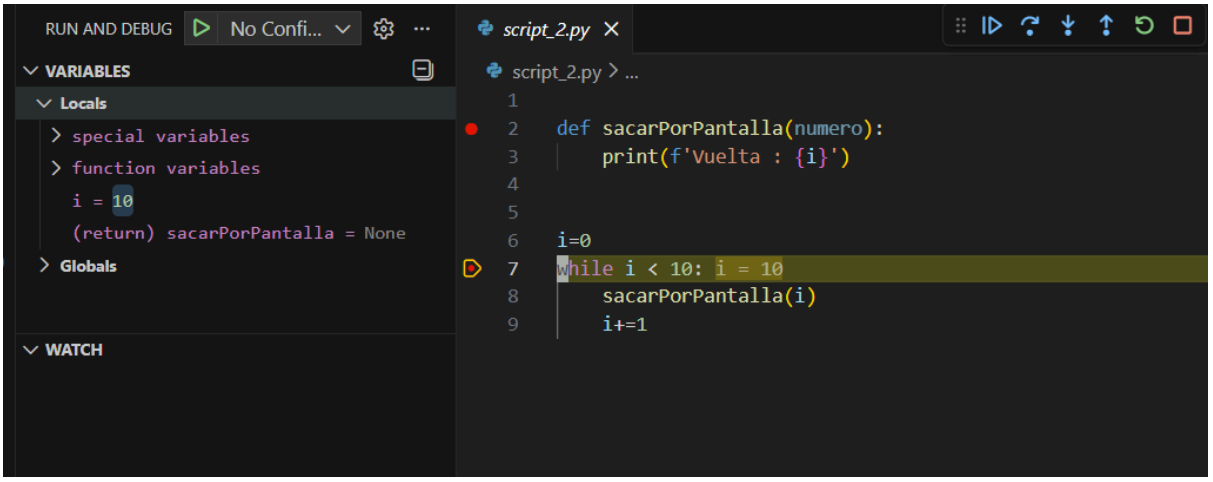
   a. <u>Continue:</u> coloqué 2 breakpoint, uno en cada función y podemos observar como al ejecutar las dos funciones analiza los valores de las dos variables y te las muestra en pantalla. Con continue, al colocar un breakpoint en cada función podemos pasar de una a la otra, sin tener que ejecutar las líneas en medio.



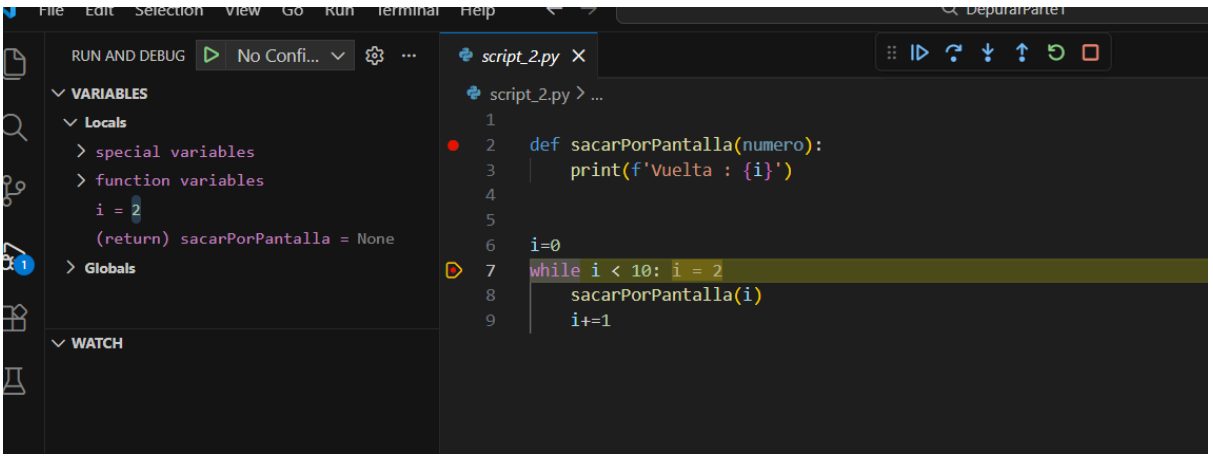


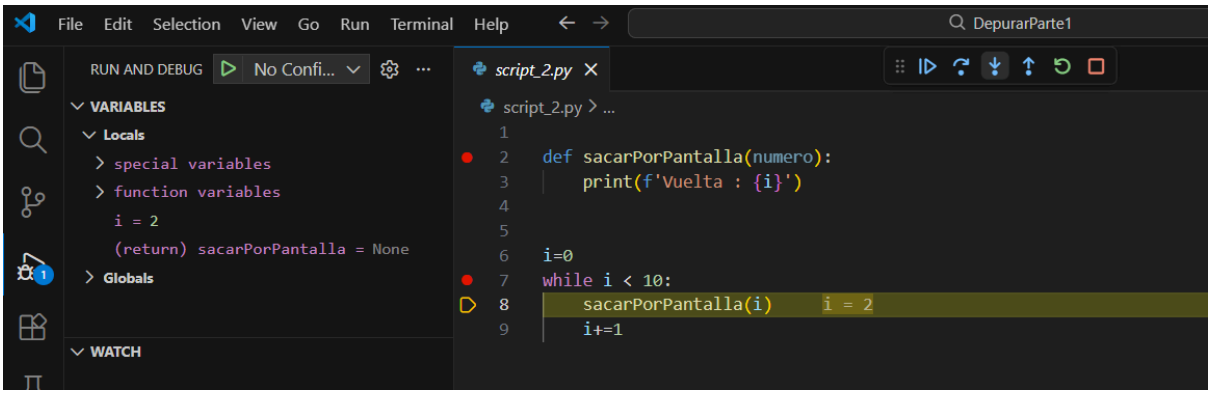
b. <u>Step over:</u> pasa de una función a otra sin entrar en print (línea 3) y analiza cada línea (captura 1) pero cuando entra en la función while, no sale hasta que el bucle, pasando por cada línea del bucle, llega a 10. Y termina el debug.

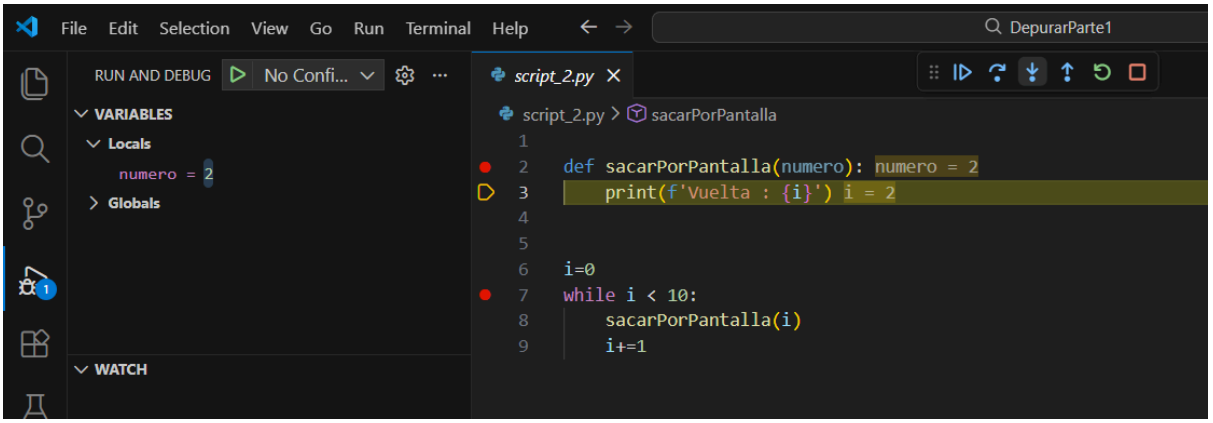


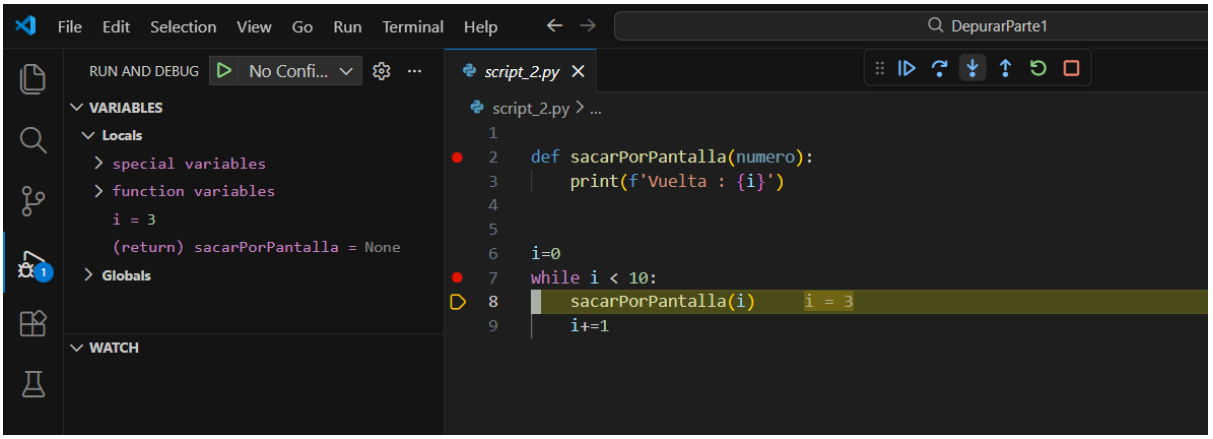


c. <u>Step into:</u> cuando usamos step into, entra en la función del while pero al ejecutar la línea 8 (captura 2), nos redirige a la variable print, y ejecuta esa variable de mostar en pantalla en vuelta del while estamos, es decir, “i=2” (captura 3), y después vuelve a la función de donde nos hemos redirigido, es decir, vuelvo a la línea 8 (captura 4).

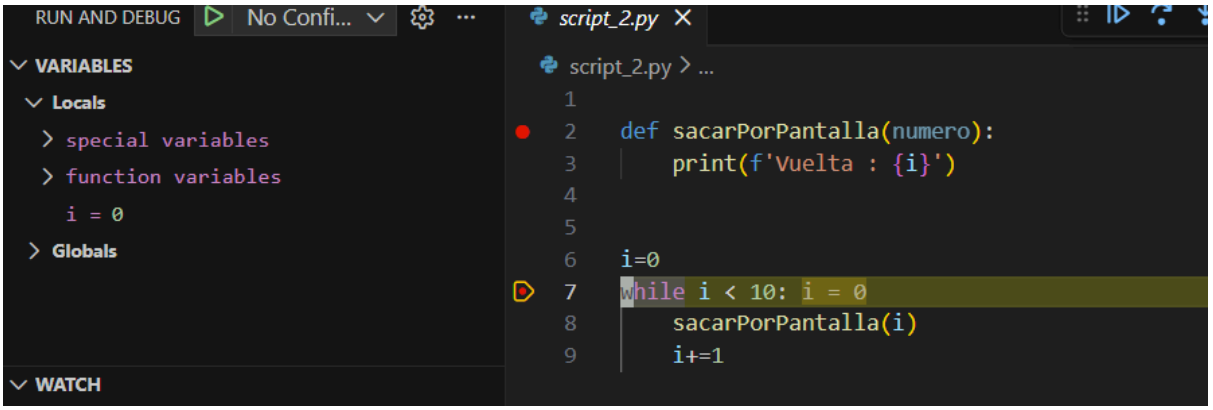


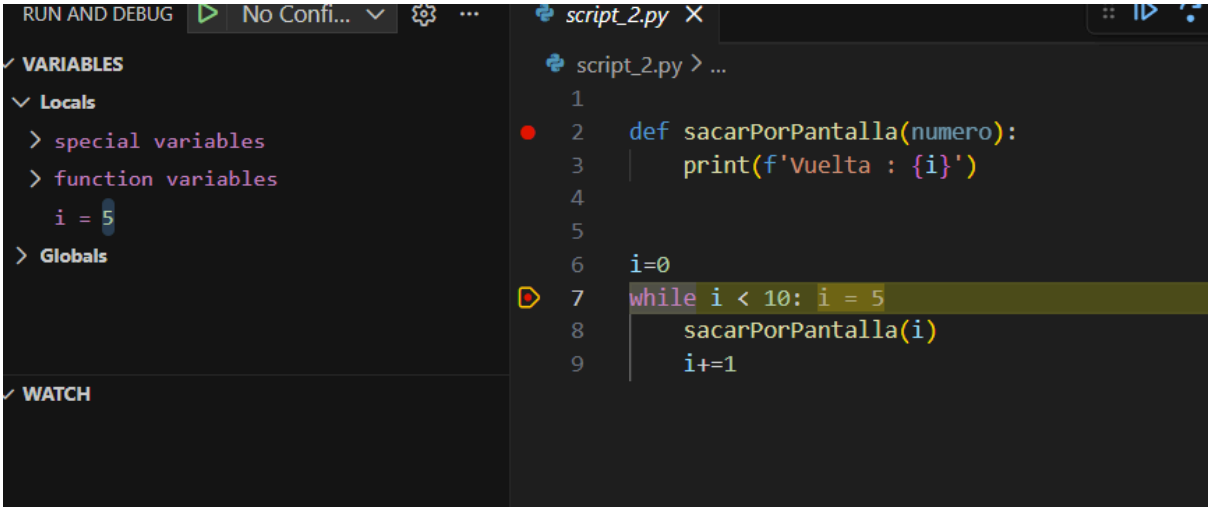






d. <u>Step out</u>: al usar step out, salto de un breakpoint a otro, sin ejecutar cada línea (captura 1), y se mantiene fijo sin moverse, en la línea del while, incrementando el valor de “i” hasta que llegue a 10 (captura 2).



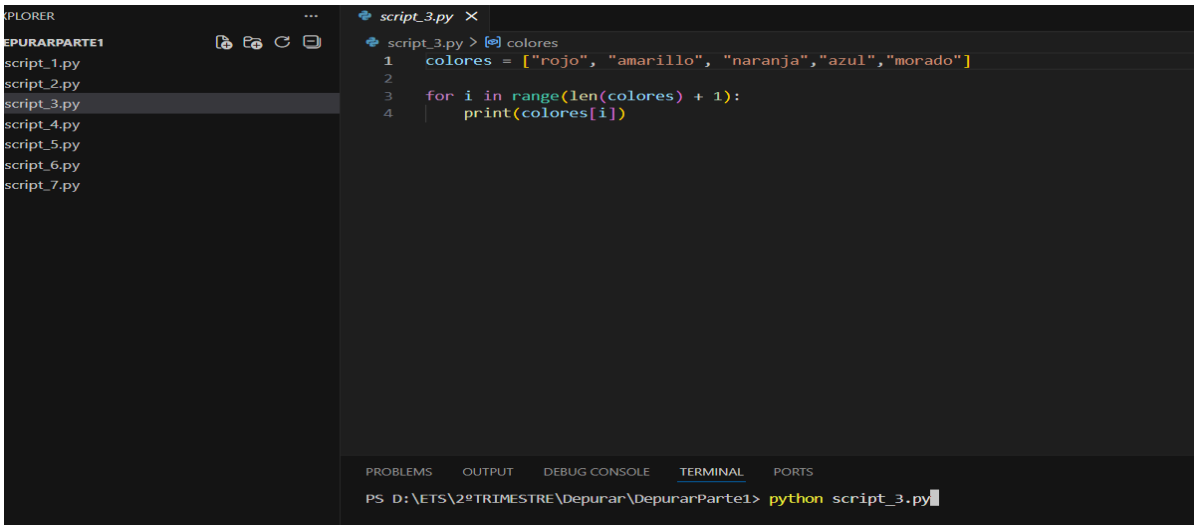


Diferencias: los únicos que saltan de un breakpoint a otro, son el continue y el step out, mientras que step into y step over, ejecutan cada línea. Otra diferencia es que solo con step into, se redirige a la línea de print y vuelve a la función para continuar el bucle.

4.3.- Ejecuta el script_3.py en modo normal.

Saca captura de pantalla de lo que ocurre. ¿Que está ocurriendo? Pon un punto de ruptura y ejecuta paso a paso para dar con el error y solucionarlo ¿Que has hecho para solucionarlo? (Adjunta capturas de pantalla de la ejecución paso a paso hasta dar con el problema y solucionarlo).

1. Nos colocamos en el script3, creamos una nueva terminal, lo ejecutamos de forma normal, sin el debug. Primero, lee los elementos que hay dentro de la array colores, es decir, lee la lista de colores que va ejecutando. Después, ocurre un error porque el código intenta acceder a una posición de la lista que no existe, es decir, que la "i" en este momento no es un valor que corresponde a una posición válida dentro de la lista de colores. Y por eso muestra este tipo de error en pantalla.



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_3.py
rojo
amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_3.py", line 4, in <module>
    print(colores[i])
          ~~~~~~~^^^
IndexError: list index out of range
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

2. Coloco un breakpoint en la línea 1 y empiezo un debug:

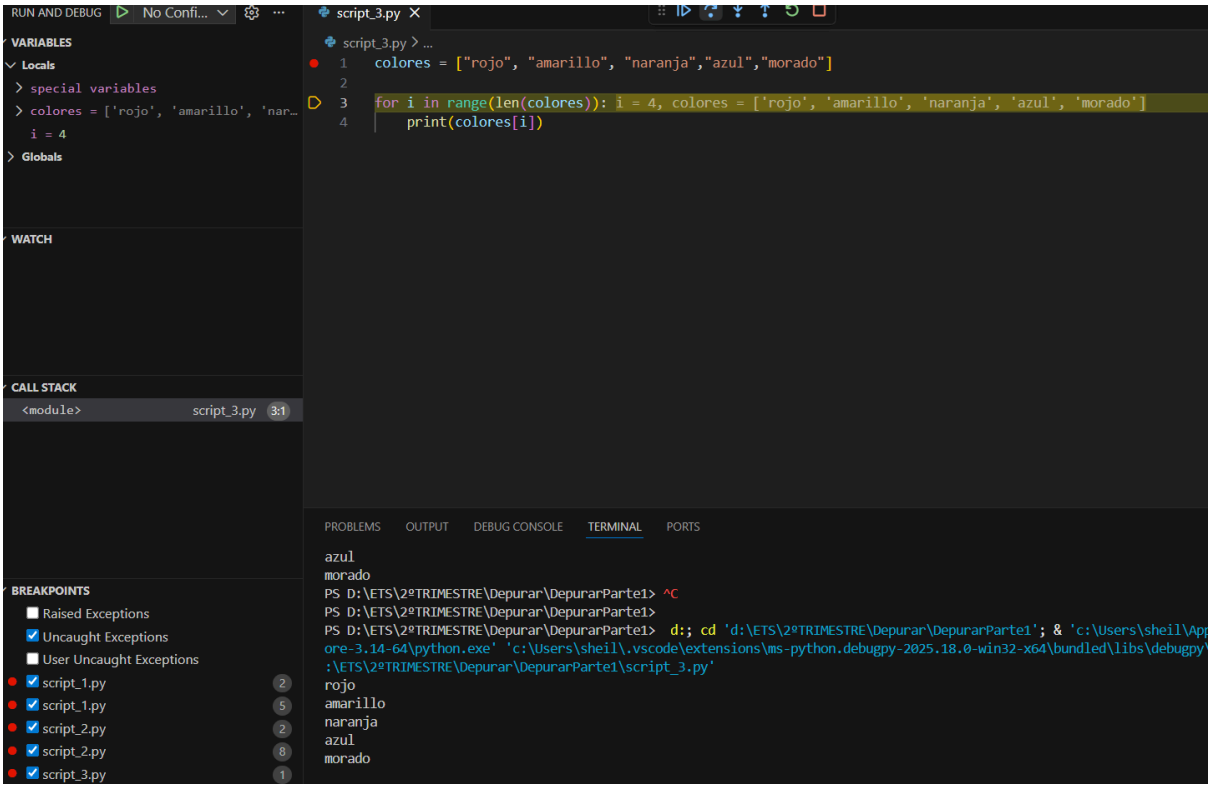
```
script_3.py
script_3.py > ...
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores) + 1):
4      print(colores[i])
```

3. Cuando hago un step into para que ejecute cada línea, por cada vuelta le suma 1 al valor de i, por cada numero, aparece en la terminal el color que corresponde a ese numero, pero cuando “i=5” aparece el siguiente error:

```
RUN AND DEBUG   No Confi...
VARIABLES
Locals
  special variables
  colores = ['rojo', 'amarillo', 'nar...
  i = 5
Globals
WATCH

script_3.py
script_3.py > ...
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores) + 1):
4      print(colores[i]) colores = ['rojo', 'amarillo', 'naranja', 'azul', 'morado'], i = 5

Exception has occurred: IndexError
list index out of range
  File "D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_3.py", line 4, in <module>
    print(colores[i])
          ~~~~~~~^^^
IndexError: list index out of range
```

Esto determina que “i” ha superado el rango que tiene la lista de colores, como la lista empieza en 0 (corresponde a rojo) cuando llega a 5, la i ha salido de la lista, porque el último color es morado que corresponde a 4.

4. Para solucionar este error, quitamos el +1 porque el rango que tiene len(colores) es de 5, pero si le sumamos 1 (como esta el codigo), intenta forzar una vuelta más, saliendo del rango que tiene la lista de colores. Por eso, al quitar esa suma, ya no existe ese error. Al terminar de recorrer la lista, termina el debug sin error.



4.4.- Ejecuta el script_4.py

¿Se produce algún error?¿Por qué? Explica que está ocurriendo.

1. Al ejecutar el script4, nos produce este error:

```
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> cd
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_4.py
rojo
Traceback (most recent call last):
  File "D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_4.py", line 5, in <module>
    resultado=colores[i]+1
              ~~~~~~~~~~~^~
TypeError: can only concatenate str (not "int") to str
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

Este error se produce porque “resultado” solo puede leer una string, pero al colocar un +1, salta el error de escritura porque estás colocando un string (colores[i]) y un int (+1).

2. Después al arreglar el primero error de escritura, nos muestra un segundo error, que es el mismo que había en el script3, de la i estando fuera del rango de la lista:

```
script_4.py ×
script_4.py > ...
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores) + 1):
4      print(colores[i]) colores = ['rojo', 'amarillo', 'naranja', 'azul', 'morado'], i = 5

Exception has occurred: IndexError ×
list index out of range
  File "D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_4.py", line 4, in <module>
    print(colores[i])
          ~~~~~~~^^^
IndexError: list index out of range

5      resultado=colores[i]
```

Para solucionar el segundo error, hacemos lo mismo que en el script3, eliminamos el +1, y ya no hay errores en el código:

```
script_4.py ×
script_4.py > ...
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores)):
4      print(colores[i])
5      resultado=colores[i]
```

4.5.- Ejecuta el script_5.py paso a paso.

Fíjate en los valores que van tomando las variables (situándote encima con el ratón y también observando en la parte izquierda en la sección de “Variables”). Comenta lo que va haciendo el código y documenta lo que ves (que tipo de ejecuciones has hecho: step over, step into,etc).

1. Al ejecutar script5
2. Salta de la línea 2, donde definía las operaciones al resultado de la operación directamente.

```
script_5.py > resultado_operacion
1
2   def operaciones(a, b):
3       resultado_intermedio = a * 2
4       resultado_intermedio_2 = resultado_intermedio*10
5       resultado_final = resultado_intermedio + b + resultado_intermedio_2
6       return resultado_final
7
8   resultado_operacion = operaciones(5, 10)
9
10  print("Resultado final:", resultado_operacion)
```

3. Al darle en step into cuando estamos en resultado de la operación, nos redirige a dentro de los pasos de la operación. Y nos sale en pantalla, los valores que tienen las variables de la operaciones, a y b (a=5, b=10).

```
VARIABLES
Locals
a = 5
b = 10
Globals
WATCH

script_5.py > operaciones
1
2   def operaciones(a, b): b = 10
3       resultado_intermedio = a * 2 a = 5
4       resultado_intermedio_2 = resultado_intermedio*10
5       resultado_final = resultado_intermedio + b + resultado_intermedio_2
6       return resultado_final
7
8   resultado_operacion = operaciones(5, 10)
9
10  print("Resultado final:", resultado_operacion)
```

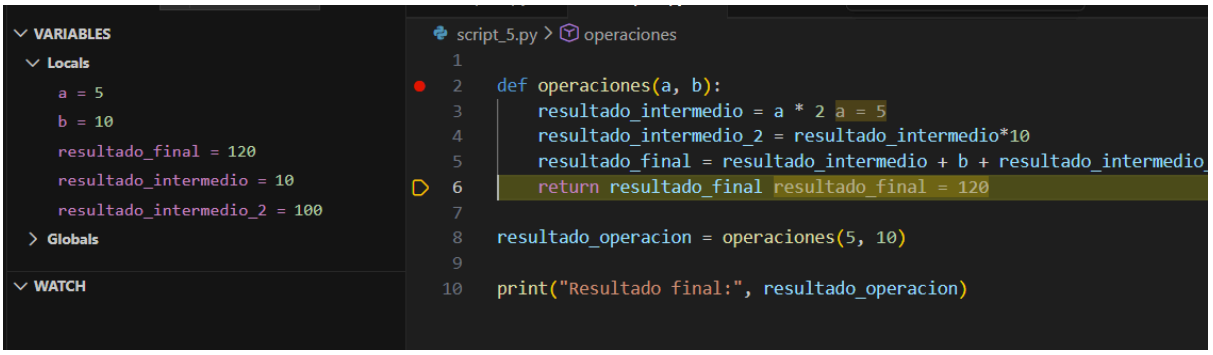
4. La segunda linea de la operación nos da el resultado intermedio que sería 10.

```
RUN AND DEBUG  No Confi...
VARIABLES
Locals
a = 5
b = 10
resultado_intermedio = 10
Globals
WATCH

script_4.py  script_5.py
script_5.py > operaciones
1
2   def operaciones(a, b): b = 10
3       resultado_intermedio = a * 2 a = 5
4       resultado_intermedio_2 = resultado_intermedio*10 resultado_intermedio = 10
5       resultado_final = resultado_intermedio + b + resultado_intermedio_2
6       return resultado_final
7
8   resultado_operacion = operaciones(5, 10)
9
10  print("Resultado final:", resultado_operacion)
```

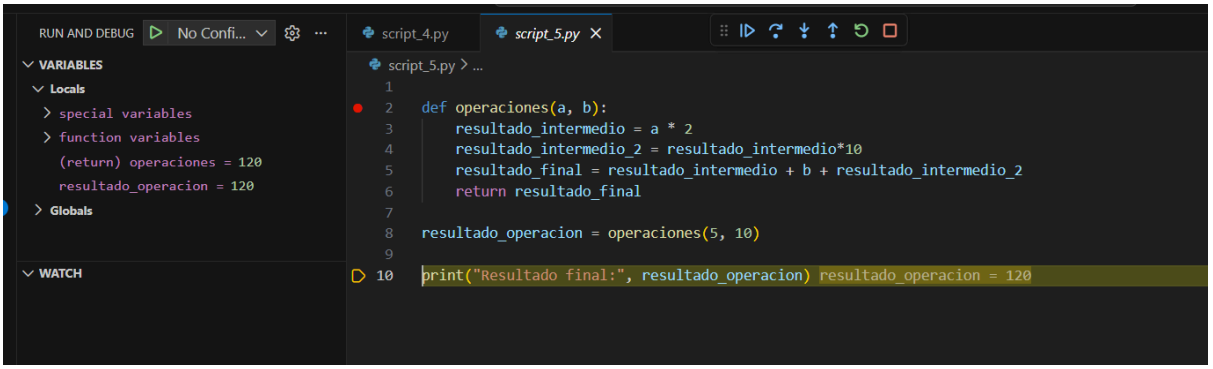
5. La tercera línea de la operación nos da el resultado intermedio 2 que es 100.

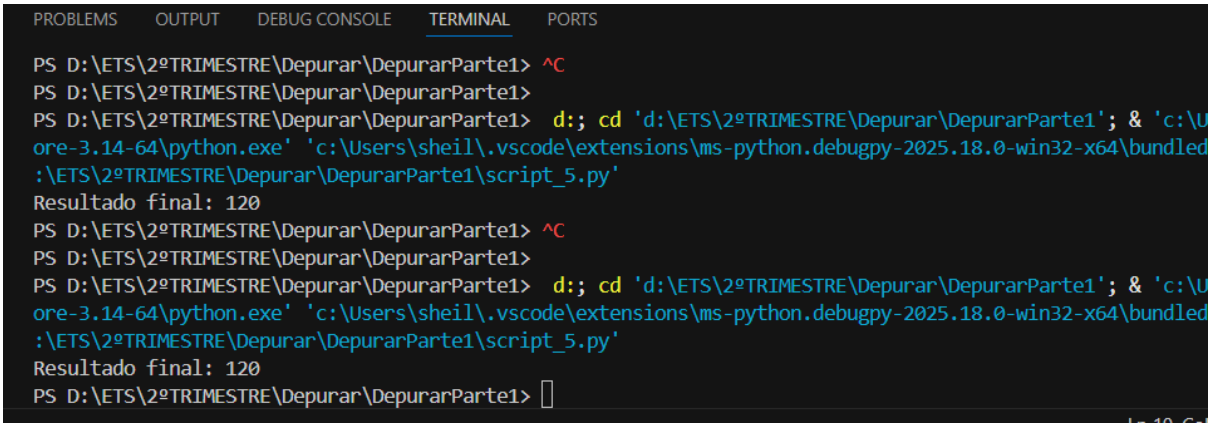
```
RUN AND DEBUG  No Confi...
VARIABLES
Locals
a = 5
b = 10
resultado_intermedio = 10
resultado_intermedio_2 = 100
Globals
WATCH

script_4.py  script_5.py
script_5.py > operaciones
1
2   def operaciones(a, b):
3       resultado_intermedio = a * 2 a = 5
4       resultado_intermedio_2 = resultado_intermedio*10
5       resultado_final = resultado_intermedio + b + resultado_intermedio_2 resultado_intermedio = 10, b = 10, resultado_intermedio_2 =
6       return resultado_final
7
8   resultado_operacion = operaciones(5, 10)
9
10  print("Resultado final:", resultado_operacion)
```

6. Y al final, nos da el resultado final de la operación: 120.



7. Después seguimos dándole step into y nos saca de la función, y podemos ver el resultado final de los valores dados y nos imprime el resultado en pantalla y en la terminal.





4.6.-Ejecuta el script_6.py.

Este script como verás, se parece al anterior con la diferencia de que es el usuario el que pasa por teclado el valor de los operandos. Prueba a ejecutar el script con varios valores. ¿Que pasa si no le metes un número y le metes letras? (Documenta el proceso como has hecho anteriormente donde quede reflejado todas las pruebas que has hecho).

1. Ejecuto el script6. Al ejecutarlo, nos pide darle los operandos para realizar después la operación.

```
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
Resultado final: 120
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> ^C
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1'; & '
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bur
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando:
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depur
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
Resultado final: 120
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> ^C
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depur
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando: 4
Introduce el segundo operando: 5
```

2. Al darle nosotros los valores, nos resuelve esta operación y nos da el resultado, que en mi caso, resulta ser 93.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2025.
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
Resultado final: 120
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> ^C
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depurar\D
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2025.
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando: 4
Introduce el segundo operando: 5
Resultado final: 93
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

3. Ahora voy a intentar ejecutarlo pero dando un valor numérico y un valor pero en forma de texto.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Resultado final: 120
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> ^C
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depurar\D
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2025.
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_5.py'
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando: 4
Introduce el segundo operando: 5
Resultado final: 93
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando: 10
Introduce el segundo operando: dos
```

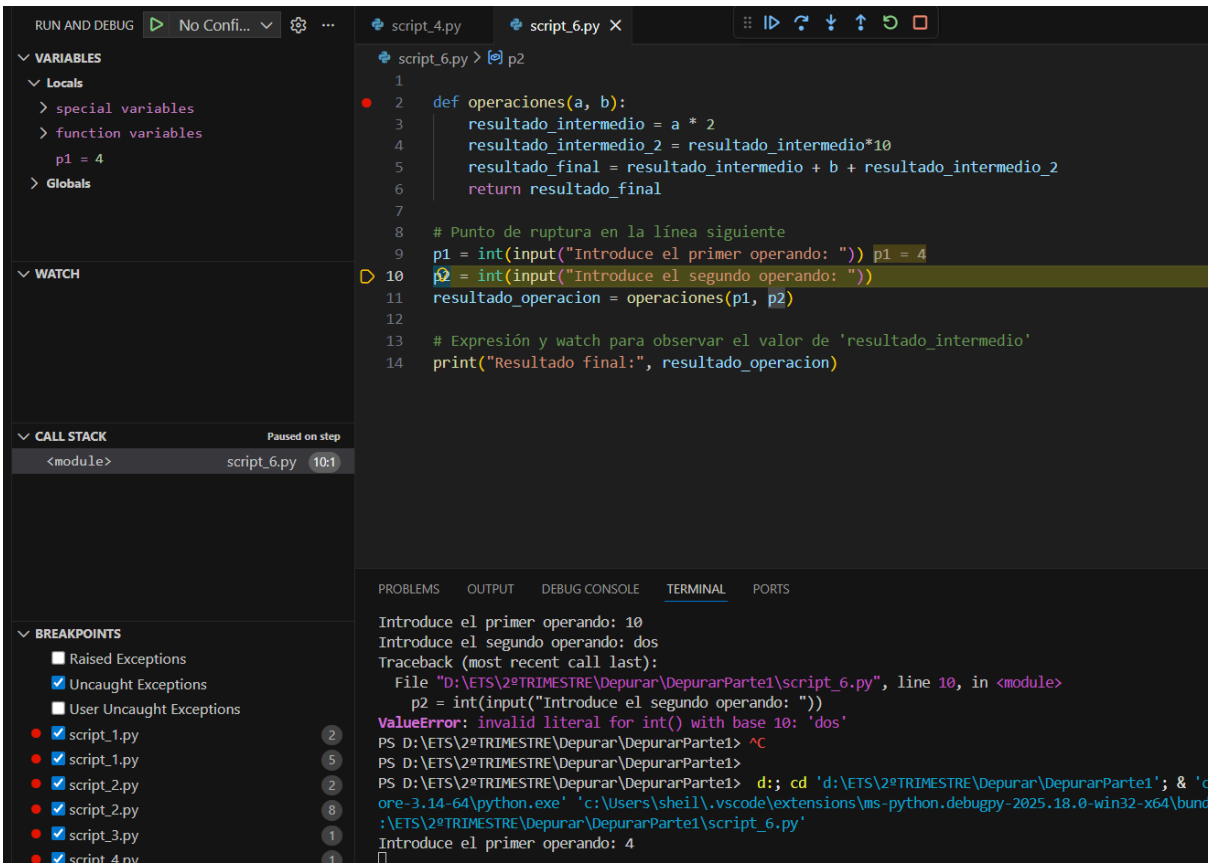
4. Al hacerlo, nos da un error de escritura, porque el código pide que le des valores numéricos y no letras.

```
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_6.py
Introduce el primer operando: 10
Introduce el segundo operando: dos
Traceback (most recent call last):
  File "D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_6.py", line 10, in <module>
    p2 = int(input("Introduce el segundo operando: "))
ValueError: invalid literal for int() with base 10: 'dos'
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

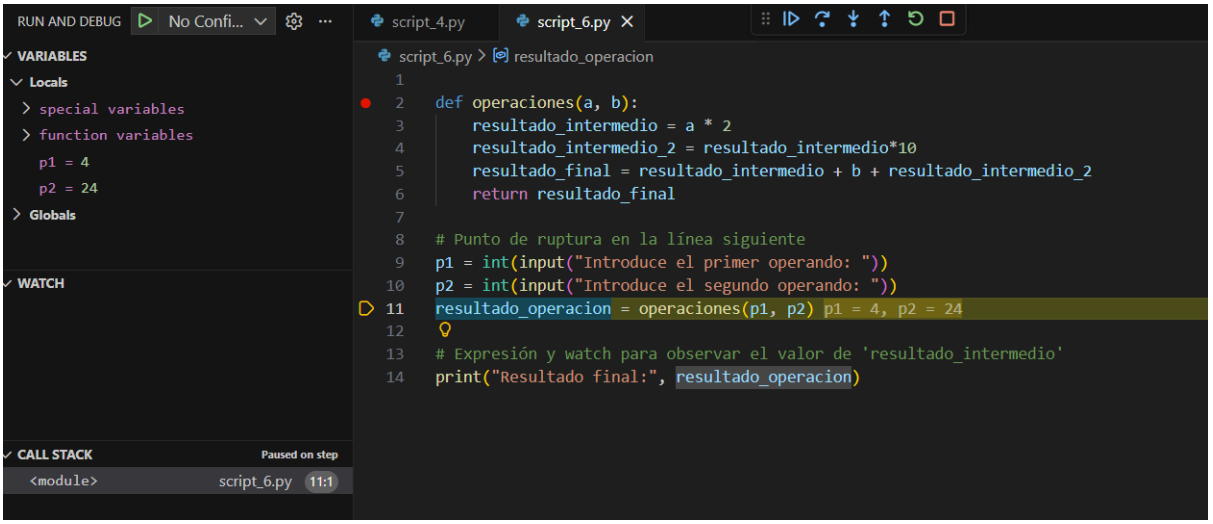
5. Ahora voy hacer todo el proceso pero en vez de con la terminal, entrando en un debug.

```
VARIABLES
  Locals
    special variables
  Globals

WATCH

script_6.py > operaciones
1
2  def operaciones(a, b):
3      resultado_intermedio = a * 2
4      resultado_intermedio_2 = resultado_intermedio*10
5      resultado_final = resultado_intermedio + b + resultado_intermedio_2
6      return resultado_final
7
8  # Punto de ruptura en la línea siguiente
9  p1 = int(input("Introduce el primer operando: "))
10 p2 = int(input("Introduce el segundo operando: "))
11 resultado_operacion = operaciones(p1, p2)
12
13 # Expresión y watch para observar el valor de 'resultado_intermedio'
14 print("Resultado final:", resultado_operacion)
```

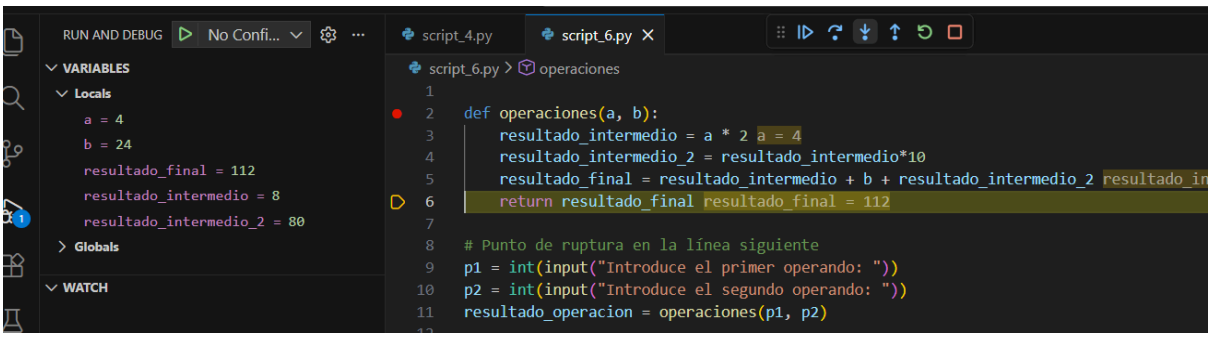
6. Utilizo el step into y nos desplaza directamente de la linea 2 a la linea 9, y nos pide en la terminal los valores que darle. Cuando coloco el valor, nos aparece en pantalla al lado de la variable a la que corresponde (p1 o p2).



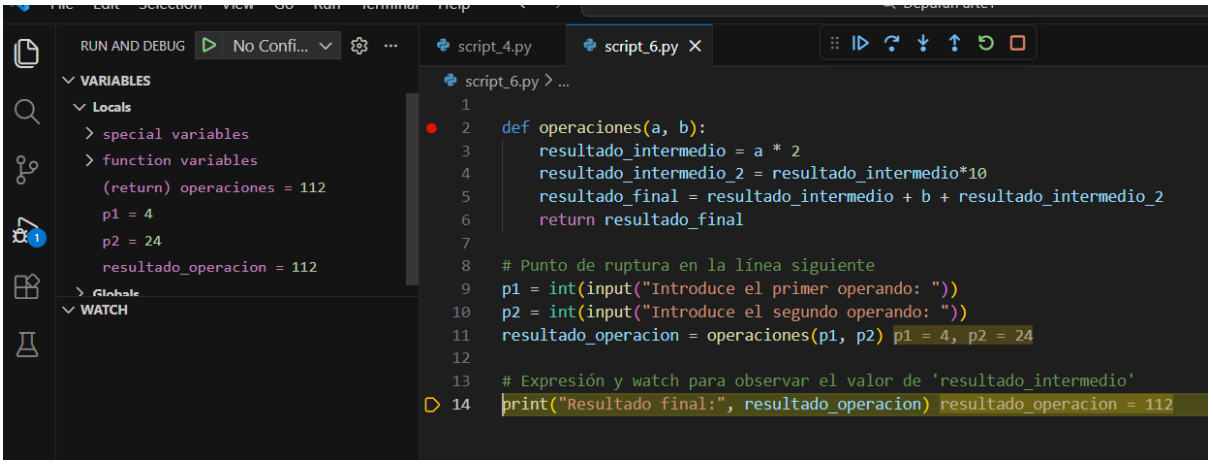
7. Para seguir la operación, le damos step into, otra vez y nos vuelve a pedir otro valor pero este corresponde a la variable p2.



8. Si después de darle los valores, seguimos utilizando step into, en vez de seguir con la siguiente línea y darnos el resultado, se redirige a dentro de la operación y podemos ver paso a paso cómo se resuelve. Estos valores, nos aparecen en pantalla según vayamos ejecutando cada línea.



9. Al acabar la operación y darnos el resultado 112. Volvemos a la línea 11, y seguimos ejecutando cada línea de código hasta que al llegar a la última línea, terminamos el debug.



4.7.- Ejecuta el script_7.py en modo normal para entender qué hace el programa.

En teoría el usuario introduce por teclado un número y el programa calcula si es un numero par o impar; después pregunta si quiere de salir de la ejecución o si quiere seguir. Si se introduce una 's' se sale de la ejecución; si se introduce una 'n' sigue pidiendo al usuario que introduzca otro número. ¿Funciona correctamente? ¿Hay algún comportamiento que no debería tener? ¿Si en lugar de poner 'y' o 'n' ponemos lo mismo pero con mayúsculas qué pasa?¿Por qué?

1. Ejecuto el script7 en modo normal:

```
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  d:; cd 'd:\ETS\2ºTRIMESTRE\Depurar\DepurarP
ore-3.14-64\python.exe' 'c:\Users\sheil\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_6.py'
Introduce el primer operando: 4
Introduce el segundo operando: 24
Resultado final: 112
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número:
```

2. Nos pide introducir un valor y después nos devuelve si es par o no. En mi caso, le di el nº 4 por lo que me devolvió que es par.

```
:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1\script_6.py'
Introduce el primer operando: 4
Introduce el segundo operando: 24
Resultado final: 112
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 4
El número es Par!
Quieres salir? y/n
```

3. Volví a intentarlo pero con un nº impar. Aunque también me pregunta si quiero salir o no pero sin importar si coloco “y” o “n”, me saca de la operación, es decir, no vuelve a poder repetirse.

```
Introduce el segundo operando: 24
Resultado final: 112
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 4
El número es Par!
Quieres salir? y/n
n
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 7
El número es Impar!
Quieres salir? y/n
y
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

4. Si coloco “s” como me dice el enunciado, me responde que debo colocar o y/n, no recoge una respuesta diferente. Pero al colocar de nuevo, y, me vuelve a pedir un numero.

```
    + FullyQualifiedErrorId : CommandNotFoundException

PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  python script_7.py
Introduce un número: 3
El número es Impar!
Quieres salir? y/n
s
Indica 'y' o 'n'
```

```
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  python script_7.py
Introduce un número: 3
El número es Impar!
Quieres salir? y/n
s
Indica 'y' o 'n' d
Indica 'y' o 'n' o
Indica 'y' o 'n'
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>  python script_7.py
Introduce un número: 3
El número es Impar!
Quieres salir? y/n
s
Indica 'y' o 'n' d
Indica 'y' o 'n' o
Indica 'y' o 'n' y
Introduce un número: 9
El número es Impar!
Quieres salir? y/n
n
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1>
```

5. Si colocamos las mismas letras epr en mayúsculas:

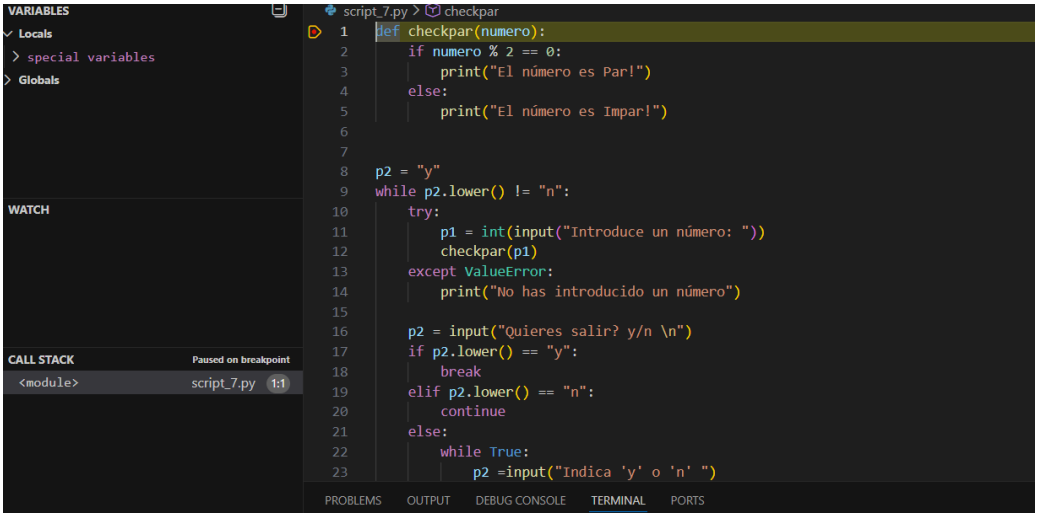
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

s
Indica 'y' o 'n' n
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 6
El número es Par!
Quieres salir? y/n
Y
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 2
El número es Par!
Quieres salir? y/n
N
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

s
Indica 'y' o 'n' n
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 6
El número es Par!
Quieres salir? y/n
Y
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
Introduce un número: 2
El número es Par!
Quieres salir? y/n
N
PS D:\ETS\2ºTRIMESTRE\Depurar\DepurarParte1> python script_7.py
```

Básicamente, el código no funciona como lo explica el enunciado. Podemos observar como s, en parte te hace repetir el valor pero primero te indica que solo puedes escribir y o n, pero la n, no te deja introducir un nº sino que te saca del bucle pero si pones y, si te vuelve a pedir el nº. En cambio, si escribimos Y o N, es decir, en mayúsculas, el código lo lee igual si está en mayúsculas o en minúsculas debido a que usamos p2.lower().

Prueba a poner puntos de ruptura y ejecutar el código línea a línea para dar con el error y solucionarlo.

6. Ahora voy a intentarlo pero en debug, poniendo breakpoints



7. He cambiado todas las “y” por una “s” porque el enunciado no habla de usar la “y” sino la “s” y donde la línea 9, cambiamos “n” por “s” que era la razón por la que no funcionaba el bucle de “n”, y te sacaba del bucle en vez de continuar. Además para que tenga lógica el código, usamos en 8, en vez de “y”, colocamos una “n” y asi el codigo solo funciona con las variables “s” y “n”.